AGENTIC AI GOVERNANCE / IMPLEMENTED DEMO

Agent Governance Framework - project handbook

Version 0.2 | 09 October 2026 | Offline, synthetic and portable

Current implementation replaces the earlier design assumptions for the same-day deadline. This document explains what is implemented and what remains future work.

CONTENTS

1. Purpose, scope and reading guide
2. What exists today
3. Installation and portability
4. Six-minute demonstration
5. Architecture and boundaries
6. Source map and technology choices
7. Identity, grants and delegation
8. Policies, data checks and exact-request approval
9. Storage, evidence and API boundaries
10. Failure handling and reset
11. Validation and its limits
12. Team roles and learning route
13. Cloud evolution and dependencies
14. Design decisions and maintenance

PROJECT REFERENCE / V0.2

1. Purpose, scope and reading guide

This handbook explains what was built, how its parts work, why the design changed for a same-day deadline, and how the team can extend it. It supersedes the initial proposed handbook. The original design is retained in the archive so earlier recommendations are not mistaken for implemented functionality.

The business problem is that agents can invoke tools, access data and delegate tasks without a consistent record of ownership, purpose and authority. The long-term goal is a cloud-neutral governance framework. The immediate deliverable is a portable local demonstration suitable for a restricted enterprise laptop, subject to that laptop's approved runtime and loopback policies.

Confirmed constraints: the working demo is needed today; there are four to five team members; Python is the strongest shared skill; the banking workflow and synthetic data are accepted; no vendor is mandatory; network and external-call permissions are uncertain; deployment into Infosys must use approved software and transfer channels. The selected production cloud remains undecided.

Read sections 2-4 first to run and present the project. Sections 5-10 explain the implementation. Sections 11-14 cover tests, operations, learning and production evolution. The application is intentionally small enough to inspect without an ML library or a frontend build system.

Implementation status must be read precisely. 'Implemented' means code exists. 'Tested' means a named scenario was executed here. Neither means a capability has been independently audited, accepted by a client or approved for the destination enterprise environment.

PROJECT REFERENCE / V0.2

2. What exists today

- A local browser dashboard with agent inventory, scenario controls, a simulated role selector, approval queue, tool execution count and event timeline.

- Three seeded agent definitions: a case coordinator, a knowledge agent and a case update agent, each with an owner, purpose, allowed actions and active/suspended state.

- A Python governance engine that authenticates opaque demo agent tokens, checks grants and policies, constrains delegation and masks responses.

- A separate tool process implementing synthetic knowledge lookup, case reads and version-checked case updates.

- Gateway-only tool access through a random per-start credential. An unauthenticated direct-tool request is rejected by the actual tool endpoint.

- SQLite persistence for control records, approvals, events, case state and tool idempotency records.

- Temporary grants, parent-grant revocation checks, one-use exact-request approvals and explicit unknown outcomes when a tool is unavailable.

- A standard-library test suite and startup scripts. The runtime needs no package downloads.

Not implemented: LLM reasoning, enterprise identity, attested workload credentials, real user entitlements, automatic discovery, registration/retirement workflows, MCP, OPA, cloud federation, multiple tenants, general text DLP, immutable logs, high availability or network isolation. The current agent definitions and policies are seeded in code. These are deliberate same-day limits, not hidden integrations.

PROJECT REFERENCE / V0.2

3. Installation and portability

Prerequisites are an approved Python 3.10+ installation with the standard SQLite module, a modern browser, a writable project folder, permission to start a Python child process, and permission to listen on local loopback addresses. No outbound internet request is needed. The destination computer's exact Python and security policy have not been verified.

On Windows, open start_demo.cmd. It changes into the project folder and uses the Python launcher when present, otherwise python. Alternatively run python app.py in the project folder. The application prints the local browser address; the default is http://127.0.0.1:8765. The tool process binds to another automatically selected loopback port. Keep the application terminal open and use Ctrl+C to stop.

Before presenting, run python app.py --check. This checks the Python version, SQLite availability and loopback binding. Then run python -m unittest discover -s tests -v to test the actual gateway/tool interaction. A successful preflight is narrower than a complete test run: it does not certify enterprise permission or all future integrations.

The distribution archive includes source, local web assets, tests and documentation. It excludes runtime databases, temporary previews, credentials and caches. Python is not bundled, because an unapproved executable may be blocked even if packaged with the project. Use the organization's approved interpreter rather than attempting to bypass those restrictions.

If the local port is busy, start with --port 8766 and use that address. If loopback or subprocess execution is prohibited, the live application cannot operate in its current form. Present the supplied architecture, test report and walkthrough and arrange an approved demo environment. Do not disable endpoint protection, use an unauthorized tunnel or upload enterprise material to a public service.

PROJECT REFERENCE / V0.2

4. Six-minute demonstration

Start with Employee selected. Explain that identities are simulated for the demonstration, while the policy, tool credential checks and data mutations execute locally. Show the scope-of-proof notice before claiming any controls.

Run Approved workflow. The coordinator receives a short-lived task grant, delegates a narrower knowledge-read task, reads the synthetic case and receives a field-filtered response. The tool execution count should increase by two. The displayed account number is masked and the raw customer field is absent.

Run Unapproved destination and Out-of-scope record. Each request is rejected before tool dispatch. Each scenario reports a tool execution delta of zero. The destination scenario never contacts an external endpoint; it supplies a forbidden destination label to the local policy engine.

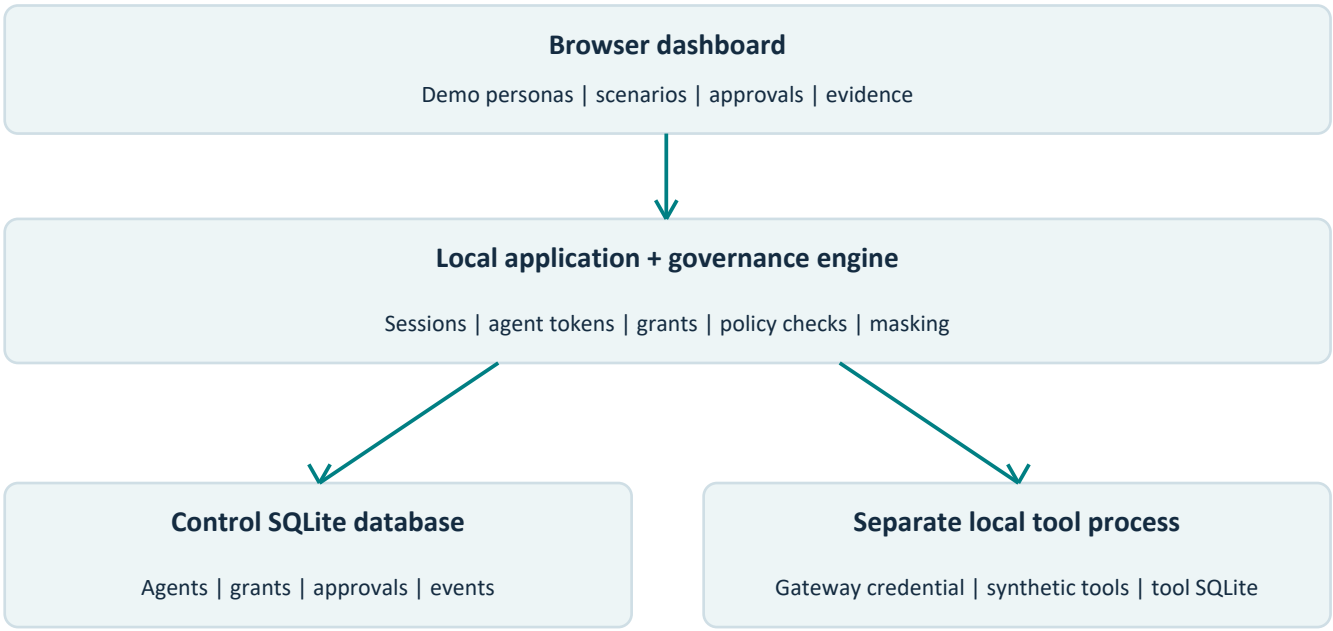
Run Privilege expansion. The knowledge agent is not eligible for case writes, so delegation fails. Run Direct-tool bypass. This sends a real loopback request to the tool service without its gateway credential; the service responds with a denial and the execution count does not change. Describe this as an application credential boundary, not network isolation.

Run Approval-gated update. The case update agent receives a narrowed grant and proposes an exact status change with the current record version. No write occurs yet. Switch to Reviewer and approve the displayed change. The case status changes and the tool execution count increases once. A second approval attempt cannot repeat the write.

Switch to Operator, run Suspend and stop descendants, and show that the child call is denied after its parent is suspended. Reactivate the coordinator in the registry before another run. Existing revoked grants remain invalid. Finish by filtering the timeline and explaining the actor, run, decision, reason and result.

PROJECT REFERENCE / V0.2

5. Architecture and boundaries



Local application checks only. No enterprise identity or network isolation claimed.

The implemented flow is Browser -> Local application/controller -> Governance Engine -> Credential-protected loopback tool service -> Synthetic case store. The governance engine writes events and control state to one SQLite database. The tool process keeps case state and execution keys in a different SQLite database. Both databases are local files under the selected data directory.

The browser is a demonstration control panel. It selects a simulated persona and invokes predefined scenarios. It never receives agent tokens or the tool credential. The local application creates an HttpOnly, SameSite session cookie and a per-session CSRF token. Those controls reduce accidental cross-site triggering, but the freely selectable persona means this is not a real login system.

Scripted agents run inside the governance process. Each has an opaque token held in memory and mapped to a seeded agent record. Their task calls are checked by the Engine. This provides testable application behavior but not process isolation between an untrusted agent and the gateway. Code running with the same local user privileges can potentially read files or process memory.

The tool service runs as a separate child process bound only to the local machine. A random credential is generated at startup and passed through the child process input pipe, not a command-line argument or browser response. Requests without it fail before invoking a tool. This does not prevent local privileged software from obtaining the credential, and loopback HTTP is not encrypted.

Production requires isolated workloads, an enterprise identity provider, workload identity, tool-side resource authorization, TLS, managed secrets, explicit network egress rules and independent operational ownership. The demo deliberately does not alter firewall rules or install system services.

PROJECT REFERENCE / V0.2

6. Source map and technology choices

app.py is the browser-facing server. make_server defines routes, session handling, host/origin checks, CSRF checks and role routing. main parses the port and data-directory options, starts the Engine, serves the dashboard and closes the child process during normal shutdown.

governance.py contains the core. ToolClient starts and calls the local tool service. Engine initializes the registry and stores. new_grant issues task authority; valid_grant checks expiry, state and ancestry; delegate narrows scope; validate evaluates concrete requests; invoke creates decisions; approve rechecks a stored exact request; execute dispatches and filters results; scenario assembles repeatable demonstrations; snapshot supplies sanitized dashboard data.

tool_server.py contains ToolStore and its HTTP adapter. ToolStore persists one synthetic case and idempotent execution results. The endpoint checks the startup credential, dispatches recognized operations and rejects stale case updates. It has no dynamic destination configuration and makes no external requests.

web/index.html defines the page; web/style.css supplies a responsive local design; web/app.js calls the API and renders results using text nodes. There are no external fonts, scripts, images or CDN dependencies. The frontend uses plain JavaScript instead of React to avoid a Node/npm installation and build step on the destination laptop. React remains a reasonable later replacement.

SQLite was selected because it is available through Python's standard library and needs no separate database service. The standard-library HTTP server was selected for an offline prototype only; Python documentation says it is not recommended for production. Python rules replace OPA in this version. NumPy, Pandas and Scikit-learn are not required because the present task is governance engineering, not model training.

PROJECT REFERENCE / V0.2

7. Identity, grants and delegation

An agent record contains a stable ID, human-readable name, owner label, purpose, permitted action list and lifecycle state. These values are seeded; the demo has no dynamic agent registration API. Owner labels represent accountable teams in the scenario and are not synchronized with a real employee directory.

Agent tokens are random values created for each application start. The Engine maps a presented token to the agent record. A request cannot choose its identity by setting an agent field. A grant includes its subject agent, allowed actions, selected case, expiry, revocation flag, delegation depth, run ID and optional parent-grant ID. The default grant lifetime is five minutes.

new_grant rejects inactive agents and scope exceeding the agent's eligibility. delegate authenticates the parent, verifies its grant, checks the child is active, rejects cycles and enforces a maximum of two delegation edges. The child's actions must fit both the parent grant and the child's own eligibility. Its expiry cannot outlive the parent because it inherits the same deadline.

Every protected call traverses the grant ancestry. An expired, revoked, missing or suspended ancestor invalidates the descendant call. Suspending an agent revokes its grants; children fail through ancestry checks. Reactivation enables fresh tasks but does not restore prior grants. The current code has no distributed cache, so this check occurs synchronously in the local process.

The requester is a fixed synthetic employee for scenario tasks. Real employee entitlements, autonomous service mandates, cryptographic workload attestation and tenant separation remain future work. The implemented grant logic demonstrates the intended structure without claiming those enterprise properties.

PROJECT REFERENCE / V0.2

8. Policies, data checks and exact-request approval

The policy version is a named constant so recorded decisions can identify the rules used. The demo's purpose is case assistance. Actions must be in the task grant, the requested case must match its scope, the destination must be internal and arguments must match a small allowlisted schema. Missing or invalid authority is denied. This is deterministic authorization, not natural-language intent inference.

Structured data protection uses field projection on case responses. The raw synthetic tool record includes a customer field and an account-like identifier. The gateway returns only the case ID, status, version and summary, plus a masking marker. The event says which field names were filtered but does not store their raw values. Arbitrary unstructured content, encoded secrets and semantic leakage are not covered by this implementation.

Case updates require approval. The stored binding includes the agent, grant, canonical request body and policy version. The body contains the target case, desired status and expected case version. A digest detects changed request content. Approval also records the requester, expiry and state. The approval lifetime is ten minutes, but the five-minute task grant must still be valid when it is consumed.

approve rejects non-reviewers, self-approval, expired or already-used approvals, changed requests and invalid current grants. It reserves the approval as executing under an application lock, then calls the tool using the approval ID as the idempotency key. The tool checks the case version and records the resulting mutation transactionally. Concurrent duplicate approvals cannot both perform the write in the tested single-process architecture.

A stale case version results in failure and a new authorization cycle is required. A timeout after dispatch is marked unknown rather than retried blindly. This matters because a remote side effect and a local approval record cannot be assumed to commit atomically. The demo does not provide automatic reconciliation of unknown writes; inspect the tool execution record before any manual retry.

PROJECT REFERENCE / V0.2

9. Storage, evidence and API boundaries

The control database has agents, grants, approvals and events tables. Grants reference parents logically; the prototype does not implement a complete enterprise schema with foreign-key migrations. Approvals hold exact request metadata and state. Events have an ordered sequence, unique ID, timestamp, event type, agent, run, decision, reason and small structured details.

The tool database has cases and executions tables. executions stores a unique execution key and its result. Reusing the same key returns the earlier result, preventing a repeated side effect. The current API is internal to the demo; production idempotency should also bind the key to an authenticated requester and request digest.

Events are committed before tool dispatch and after observed outcomes. The browser shows and exports the most recent 150 events; the database retains the full local history. This is durable local evidence, not an immutable audit store. A local user with filesystem access can modify it. There is no automatic retention deletion, audit forwarding, signed log chain or protected archive in this build.

GET /api/health reports readiness. POST /api/session selects a demo persona. GET /api/state returns sanitized dashboard state. POST /api/scenario runs one predefined workflow. POST /api/approve submits an approval ID. POST /api/suspend and /api/reactivate change seeded lifecycle state. GET /api/export returns recent evidence. The tool service accepts only POST /invoke with its separate gateway credential.

The public browser API intentionally does not expose raw agent invocation, arbitrary target URLs or credential issuance. The tests call core methods directly to exercise invalid credentials and altered requests. Authentication failures, cross-origin requests and role errors are covered by HTTP tests. Session selection itself is a demo facility, not proof of human authentication.

PROJECT REFERENCE / V0.2

10. Failure handling and reset

If a grant expires during presentation, run the scenario again to obtain fresh authority. If an approval refers to an expired grant, it cannot extend the task; create a new approval request. Pending approvals may still display as pending until attempted, but their execution-time checks reject expired authority.

If the coordinator was suspended, choose Operator and reactivate it. Then start a new scenario. If all operations fail because the tool process stopped, stop and restart the application. A tool unavailable after dispatch produces an unknown result in the core; the dashboard may report local-service unavailability when it cannot retrieve tool statistics.

On application restart, old grants are revoked, pending approvals are expired, interrupted executing approvals become unknown, and agent/case history remains. This conservative reset avoids silently resuming an authorized action after its original runtime context has disappeared.

For a clean presentation, stop the application and use a new data directory. This preserves previous evidence rather than deleting it. Avoid running two application instances against the same data directory. The prototype uses an in-process lock and was not designed for multiple gateway workers sharing the same state.

Normal Ctrl+C shutdown closes the server and child process. An abrupt process kill may leave a child tool process running; use the organization's approved process-management tools or restart the machine according to policy. Do not expose either server to the LAN or public internet.

PROJECT REFERENCE / V0.2

11. Validation and its limits

The standard-library unittest suite exercises the actual Engine and a real separate tool process, using temporary databases. The recorded run passed 25 tests. They cover masked allowed reads, unknown identity, case scope, forbidden destination, delegation expansion, expiry, parent suspension, wrong grant subject, purpose/schema validation, direct-tool rejection, one-use approval, reviewer rules, request tampering, revocation recheck, stale version, concurrent approvals, tool idempotency, cycles, reactivation, operator role and tool outage handling.

HTTP tests verify local static assets, required sessions and CSRF, rejected cross-origin session creation and the inability to obtain operator behavior merely by adding a role field to the request body. Browser walkthrough results are recorded separately in TEST_REPORT.md so automated tests are not confused with visual verification.

No penetration test, production load test, regulatory assessment, Infosys deployment test, real cloud integration test or LLM evaluation has been performed. A successful direct-tool denial demonstrates that endpoint's credential check; it does not prove that an agent process cannot reach it over the network or steal a credential from the same machine.

The source is the ultimate implementation reference. If a future change modifies state transitions, grant scope, policy inputs, approval binding, retry semantics or UI permissions, update the corresponding tests and this handbook in the same change.

PROJECT REFERENCE / V0.2

12. Team roles and learning route

For four people: one owns Python governance and policies; one owns tool/identity adapters and deployment; one owns the dashboard and usability; one owns testing, documentation and the demonstration narrative. With a fifth person, separate security review and integration research from testing. These are suggested responsibilities, not named assignments.

First learn Python dictionaries, functions, exceptions, modules, JSON and HTTP requests. Then trace one allowed workflow through scenario, delegate, invoke, validate and execute. Inspect the events after each stage. Change a test input to see how a denial reason connects to a rule. Use synthetic data throughout.

Next learn SQLite transactions, locks, idempotency and optimistic record versions. These explain why approval is more than a button. Then study browser fetch, safe text rendering, cookies and CSRF using the small frontend. ML libraries are not prerequisites for these lessons.

Later learn OAuth/OIDC, workload identity, policy engines, container networking and MCP. Add an agent framework only after the gateway contract is stable. A model should produce a proposed action; the gateway must still decide whether that action is authorized.

PROJECT REFERENCE / V0.2

13. Cloud evolution and dependencies

Stage A is today's local prototype. It needs only the approved interpreter, browser and loopback. Stage B is a controlled sandbox: replace simulated personas with enterprise SSO, bind isolated workloads to verified identities, move persistent state into managed PostgreSQL, use a production Python service stack such as FastAPI, and deploy behind approved TLS ingress. Preserve tests for existing policy behavior while changing adapters.

Stage C introduces production enforcement boundaries: isolate agent workloads, deny direct unapproved egress, make protected tools verify destination-specific authority, manage secrets/keys centrally, and establish policy review and audit retention. Validate suspension propagation and outage behavior. These changes require platform and security approvals; the current archive makes no firewall or cloud changes.

Stage D adds a real agent framework, an approved model endpoint and selected MCP/HTTP integrations. Tool discovery does not imply trust. Pin tool schemas, map each operation to policy, classify request/response fields and prevent raw token passthrough. The deterministic scenarios remain as regression fixtures even after real model calls are introduced.

Cloud services to decide: container/application hosting, identity, secrets/key management, relational database, private networking and ingress, object/log storage, monitoring, build registry and the model service if used. Azure, AWS and GCP can all supply these categories; the actual provider should follow the client's landing zone and approvals. No provider credentials, paid resources or deployment are included.

Before a pilot, agree data region and retention, support owners, expected traffic, availability, recovery objectives, user entitlements and incident workflow. Confirm commercial licenses and dependency policies during stack selection. The reported application count in the source material is not a benchmark result or sufficient capacity model.

PROJECT REFERENCE / V0.2

14. Design decisions and maintenance

D01: standard-library Python instead of installing a larger stack today. Benefit: fewer destination-machine prerequisites. Cost: the built-in HTTP server is local-demo-only and must be replaced before deployment.

D02: plain local browser assets instead of React today. Benefit: no Node or package-build requirement. Cost: less frontend structure for a large product. The backend contracts can support React later without changing the governance model.

D03: deterministic agents and synthetic tools instead of a live model. Benefit: repeatable results and no data leaving the laptop. Cost: this does not demonstrate autonomous reasoning or real MCP interoperability.

D04: a separate tool process with a random startup credential. Benefit: the bypass demonstration exercises an actual endpoint. Cost: both services remain under the same local user and there is no network isolation or attestation.

D05: explicit role simulation. Benefit: the panel can see employee/reviewer/operator behavior without credentials or enterprise accounts. Cost: users may select any persona, so no claim of real separation between authenticated humans is valid.

D06: preserve the initial PDFs as historical design, maintain this handbook as current behavior, and include tests in the portable package. Every future release should record changed requirements, implementation files, dependency versions, test results, known gaps and operational changes. Mark features proposed, implemented or verified; never silently upgrade a proposal into a claim.

References consulted: Python http.server and sqlite3 documentation; the initial implementation plan links official MCP, IETF OAuth token exchange, SPIFFE, OPA, OpenTelemetry, FastAPI, Keycloak, Kubernetes and Docker documentation. Those sources inform the future architecture; they do not imply these components are installed in this demo.